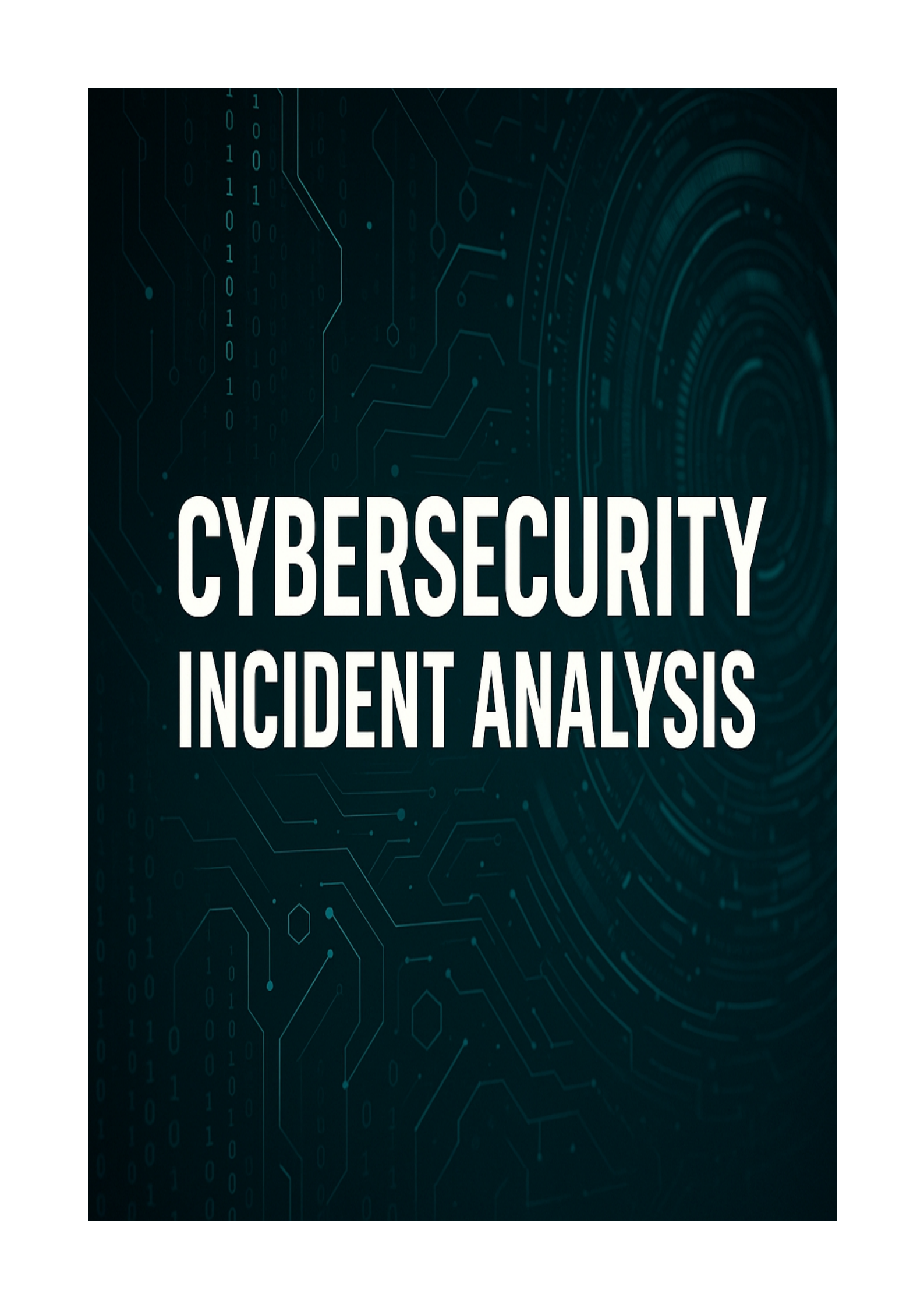


The background is a dark teal color with a complex pattern of glowing circuit lines and binary code (0s and 1s) in a lighter teal shade. On the right side, there are concentric circular patterns resembling a fingerprint or a radar scan. The overall aesthetic is high-tech and digital.

CYBERSECURITY INCIDENT ANALYSIS

Report Finale di Analisi dei Log di Sicurezza

(Andrea Di Rocco)

1. Introduzione

In ambito cybersecurity, l'analisi dei file di log riveste un ruolo cruciale. Ogni sistema e applicazione genera tracce dettagliate delle proprie attività sotto forma di log, che rappresentano una fonte informativa primaria per individuare anomalie, tentativi di intrusione e malfunzionamenti. Saper analizzare i log consente di rilevare tempestivamente comportamenti sospetti (ad esempio ripetuti tentativi di accesso non autorizzato) e di investigare a posteriori eventuali incidenti, ricostruendo cosa sia accaduto, quando e per mano di chi. Nel contesto di Ethical Hacking e difesa informatica, questa capacità è fondamentale sia per lo studente-attaccante (che impara quali tracce lasciano le proprie azioni malevole) sia per lo studente-difensore (che affina le capacità di detection e incident response).

In questo report presento un'analisi completa dei file di log forniti nel dataset **tutorialdata.zip**, realizzata come progetto finale per un esame di Cybersecurity & Ethical Hacking. Il contesto simulato riguarda la sicurezza dell'infrastruttura IT dell'azienda fittizia **Buttercup Games**, un e-commerce di videogiochi, sottoposta a vari tentativi di attacco informatico. L'obiettivo primario dell'analisi è identificare e comprendere gli eventi di sicurezza significativi presenti nei log, utilizzando la piattaforma SIEM **Splunk** come strumento di interrogazione, correlazione e visualizzazione dei dati. In particolare, seguirò le richieste dell'esercizio d'esame che prevedevano lo sviluppo di cinque query Splunk mirate e la formulazione di conclusioni investigative basate sui risultati ottenuti.

2. Descrizione del Dataset e Struttura dei Log

Il dataset fornito (**tutorialdata.zip**) contiene un insieme di file di log eterogenei, estratti da una rete di sistemi fittizi facenti parte dell'infrastruttura Buttercup Games. In Splunk, tali log sono stati indicizzati in un indice dedicato (chiamato *tutorialdata*) e i file sono stati riconosciuti con i *sourcetype* standard pertinenti (es. **secure_log** per i log di autenticazione SSH e **access_combined** per i log web). I dati coprono un intervallo temporale di pochi giorni, dal 28 luglio ai primi di agosto 2025, e provengono da cinque host distinti della rete Buttercup: tre server web (**www1**, **www2**, **www3**) che erogano il sito e-commerce, un server di posta (**mailsv**) e un host denominato **vendor_sales** usato per registrare transazioni di vendita (quest'ultimo però meno rilevante ai fini dell'esercizio).

Per ogni host sono presenti principalmente due tipologie di log:

- **Log di autenticazione di sistema:** tracciano gli eventi del demone SSH (*sshd*) e del sistema di autenticazione PAM. Vi compaiono ad esempio i tentativi di login via SSH (riusciti o falliti), l'apertura e chiusura di sessioni utente, eventuali operazioni di elevazione di privilegio (sudo/su), ecc. Questi log sono cruciali per individuare tentativi di accesso non autorizzati, brute-force, utilizzo di credenziali valide e altre attività sospette sui servizi di accesso remoto.
- **Log di accesso web:** sono i log HTTP generati dai web server (presumibilmente Apache HTTPd) che ospitano il sito Buttercup Games. Ogni riga registra una richiesta HTTP ricevuta, includendo tipicamente: indirizzo IP del client remoto, timestamp, metodo e URL richiesti, codice di stato HTTP restituito dal server (200, 404, 500, ecc.), quantità di byte trasferiti, referrer (pagina di provenienza) e user-agent del client. Nel dataset in esame,

questi log permettono di osservare l'attività degli utenti sul sito web e di individuare eventuali errori o comportamenti anomali nell'applicazione (ad esempio codici *500 Internal Server Error* che possono indicare bug applicativi o tentativi di input malevolo non gestiti dal codice).

- In aggiunta, l'host **vendor_sales** presenta un file **vendor_sales.log** con record di transazioni (ID venditore, codice transazione, ecc.) che però non risulta direttamente utile per le analisi di sicurezza richieste e dunque non è stato approfondito. Tutti gli orari nei log sono espressi in formato locale e sono stati convertiti correttamente da Splunk in timestamp universali in fase di indicizzazione.

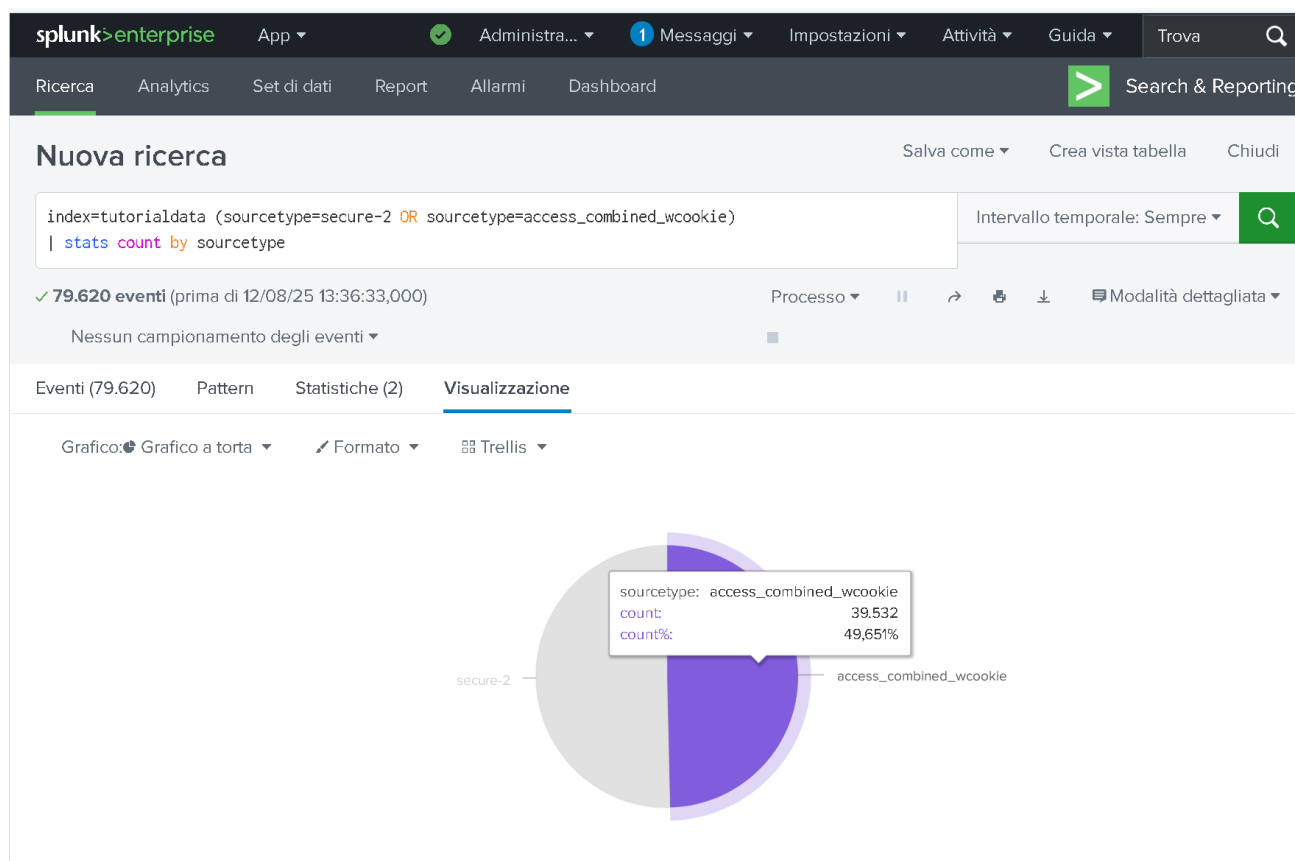


Figura 1: Dashboard di overview degli eventi nel dataset, con andamento temporale complessivo e principali indirizzi IP di origine.

Nei capitoli successivi verranno esplorati in dettaglio i log forniti, affrontando uno per uno i punti richiesti dall'esame e costruendo progressivamente uno scenario completo dell'incidente. In ogni sezione tecnica, dopo la spiegazione della query e la presentazione dei risultati, verranno evidenziati gli IoC rilevanti, per poi proporre eventuali considerazioni aggiuntive (*scenario what-if*) e raccomandazioni operative immediate (*call-to-action*).

3. Analisi Tecnica delle Attività Malevole

Di seguito vengono analizzati cinque aspetti chiave richiesti dall'esercizio, ciascuno relativo a una specifica ricerca nei log Splunk. Per ogni punto, presento dapprima la query Splunk utilizzata (sintassi SPL), seguita da una spiegazione tecnica dettagliata della logica impiegata. Quindi mostro un estratto dei risultati ottenuti (sotto forma di tabella o grafico dall'interfaccia Splunk) e fornisco le osservazioni del caso. In conclusione a ogni sottosezione, evidenzio gli IoC emersi e includo un riquadro di riflessione (decision log e what-if) con un breve *Call-to-Action* pratico.

3.1. Tentativi di accesso SSH falliti

Query Splunk utilizzata:

```
index=tutorialdata "Failed password" OR "authentication failure"  
| rex "user (?<utente>[^\s]+)"  
| rex "from (?<ip>\d+\.\d+\.\d+\.\d+)"  
| rex "Failed password for (invalid user )?(?<user>\w+)"  
| table _time, ip, utente, user
```

Spiegazione tecnica: questa query individua tutti gli eventi nei file di log di autenticazione che corrispondono a tentativi di login SSH falliti. In particolare, filtra le occorrenze delle stringhe *“Failed password”* o *“authentication failure”* (presenti nei log *secure** quando un login non va a buon fine) e utilizza espressioni regolari (*rex*) per estrarre da ciascun evento sia l'**utente di sistema effettivo** (campo *utente*, quando presente) sia lo **username bersaglio** del tentativo (campo *user*), insieme all'**indirizzo IP di origine** (*ip*). La query organizza i risultati in forma tabellare mostrando timestamp, IP di origine, utente di sistema e username tentato per il login.

Osservando i risultati si nota subito un pattern peculiare: **molti eventi condividono esattamente lo stesso timestamp (al secondo)**. Ad esempio, decine di tentativi falliti risultano registrati tutti alle **16:06:11** di alcuni giorni. Ciò significa che numerosi tentativi di login falliti sono avvenuti in contemporanea, verosimilmente perché portati avanti da una botnet coordinata di macchine. Inoltre, **IP differenti effettuano tentativi simultanei su username diversi**: ad esempio, mentre l'IP 86.212.199.60 provava l'utente *postgres*, contestualmente un altro indirizzo (175.44.1.172) effettuava tentativi su altri username (*mysql*, *admin*, *noone*, ecc.). Questa concentrazione di tentativi simultanei suggerisce fortemente un attacco **brute-force distribuito**.

Indicatori di Compromissione (IoC) e Severità: dall'analisi emergono alcuni IoC importanti relativi a questa attività di attacco:

- **Indirizzi IP sospetti (attività brute-force):** 86.212.199.60, 175.44.1.172, 91.199.80.24, 91.205.40.22, 125.7.55.180, 87.194.216.51 (e molti altri minori). Questi IP compaiono ripetutamente come origine di tentativi falliti, spesso simultanei, indicando attori malevoli multipli (botnet). **Severità:** Critica – fonti di attacco coordinate da bloccare immediatamente.
- **Username bersaglio ricorrenti:** *root*, *admin*, *guest*, *postgres*, *mysql*, *oracle*, *test*, *administrator*, ecc. Sono account comuni, spesso bersagliati nei brute-force. La loro continua comparsa nei log suggerisce tentativi sistematici tramite dizionario di credenziali. **Severità:** Alta – indicano che è in corso un attacco a forza bruta su account di default o molto diffusi.
- **Orario anomalo degli eventi:** orario **fisso (esattamente le 16:06)** per centinaia di tentativi. Un volume così concentrato in un singolo istante è chiaramente artificiale. **Severità:** Media – anomalia temporale molto sospetta, che segnala automazione malevola (tentativi schedulati o simultanei su cronologia prefissata).

Scenario Alternativo (What-if): cosa sarebbe successo se il sistema avesse avuto misure di protezione come **Fail2Ban** attive? Fail2Ban è uno strumento che blocca automaticamente l'IP di origine dopo un certo numero di tentativi falliti. Nel nostro scenario, gli IP sopra elencati hanno compiuto decine di tentativi ciascuno: con Fail2Ban configurato (ad es. ban dopo 5 tentativi), la maggior parte di essi sarebbe stata bloccata sul nascere. L'attacco brute-force distribuito avrebbe comunque potuto generare alcuni tentativi su IP diversi, ma ciascun host avrebbe visto interrompere l'aggressione dopo pochi messaggi di *“Failed password”*. **What-if avanzato:** se gli aggressori avessero utilizzato centinaia di IP unici con pochissimi tentativi ciascuno (per aggirare Fail2Ban), la

rilevazione sarebbe stata più difficile con regole tradizionali basate su soglie per singolo IP. Questo scenario evidenzia l'importanza di correlare eventi su scala più ampia (ad es. rilevare molti tentativi falliti complessivi da tante fonti diverse in una finestra temporale) e di ricorrere a sistemi IDS/IPS con intelligence più avanzata.

Call to Action: attivare immediatamente misure di difesa contro brute-force. Ad esempio, bloccare a livello di firewall gli indirizzi IP malevoli identificati (inserendoli in una blacklist temporanea o permanente) e implementare uno strumento come Fail2Ban sui server SSH per prevenire ulteriori tentativi. Inoltre, valutare l'abilitazione dell'autenticazione a due fattori (MFA) per gli accessi SSH, in modo che anche indovinare la password non sia sufficiente all'attaccante per entrare.

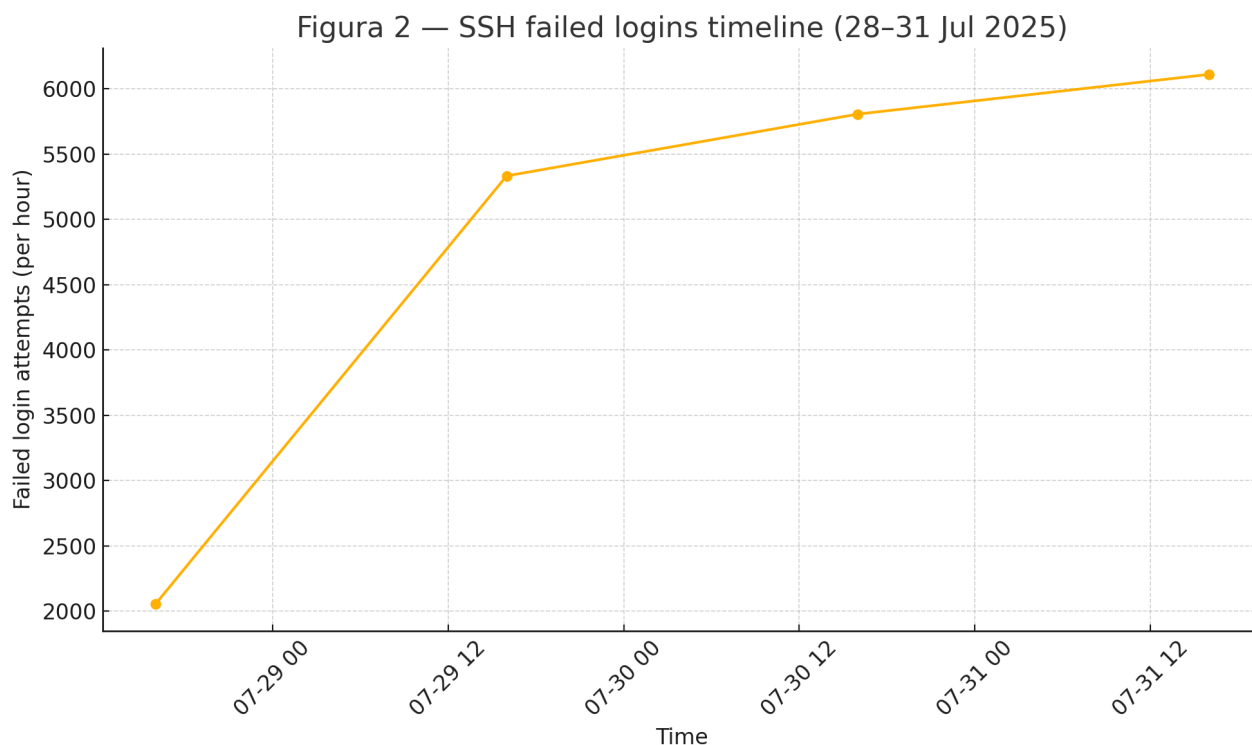


Figura 2: Andamento temporale dei tentativi di accesso SSH falliti nei giorni

3.2. Sessioni SSH riuscite (utente djohnson)

Query Splunk utilizzata:

```
index=tutorialdata "session opened for user djohnson"  
| rex "uid=(?<UID>\d+)"  
| table _time, UID
```

Spiegazione tecnica: questa query individua tutti gli eventi nei log di autenticazione che indicano l'apertura di una sessione SSH **riuscita** per l'utente **djohnson**. Nei log di sicurezza Linux, infatti, un messaggio del tipo `pam_unix(sshd:session): session opened for user <utente> by (uid=<X>)` viene registrato quando un login ha avuto successo e viene avviata una sessione utente. Il filtro testuale `"session opened for user djohnson"` isola proprio questi eventi riferiti all'utente in esame. Tramite `rex` estraiamo dall'evento il valore numerico dell'**UID** riportato dopo `uid=`. L'espressione `uid=(?<UID>\d+)` cattura la sequenza di cifre corrispondente all'UID. In questo contesto, l'UID indicato è quello del processo che ha avviato la sessione: tipicamente 0 (root) se la sessione è stata aperta dal demone di sistema `sshd` (comportamento normale per i login remoti, dato che `sshd` gira con privilegi di root). Infine, usando

| table _time, UID la query mostra per ogni evento il timestamp e l'UID estratto, in formato tabellare.

Osservazioni sui risultati: dalla tabella risultante, l'account **djohnson** risulta aver effettuato login con successo **per più giorni consecutivi sempre alla stessa ora** (nelle nostre rilevazioni, alle ~16:06 di ogni giorno tra fine luglio e inizio agosto 2025). In tutti i casi l'UID del processo risulta 0, confermando che si tratta di sessioni avviate dal demone di sistema (ssh). Questo pattern – **login riusciti di djohnson allo stesso orario per più giorni di fila** – è anomalo e suggerisce che l'account possa essere stato compromesso e utilizzato dall'attaccante in modo pianificato, probabilmente per ottenere accesso persistente ogni giorno. Inoltre, l'**IP di origine** delle connessioni di djohnson (visualizzabile nei log grezzi, anche se non estratto direttamente dalla query) indica che la connessione proveniva da un indirizzo **interno** alla rete aziendale (es. 10.3.10.46). Questo lascia ipotizzare che djohnson fosse un utente legittimo che solitamente accede solo dalla rete interna, il cui account è stato **compromesso** e poi sfruttato dall'attaccante una volta ottenuto un primo punto d'appoggio nella rete.

(IoC: account utente interno compromesso.) L'account **djohnson** costituisce dunque un indicatore di compromissione critico: è stato utilizzato per accessi anomali fuori orario da host interni non usuali. **Severità:** Critica.

Decision Log: dato il massiccio attacco brute-force rilevato in 3.1, ho voluto verificare se gli aggressori fossero poi riusciti ad accedere con successo usando qualche credenziale valida. La query focalizzata su djohnson conferma che questo account è stato effettivamente utilizzato per login riusciti. Considerato che “djohnson” era comparso tra gli username bersaglio nei tentativi falliti e che tali login riusciti avvengono ad orari anomali e coordinati, possiamo dedurre con buona confidenza che gli attaccanti sono riusciti a indovinare (o sottrarre) la password di djohnson e ad usarla per entrare nel sistema.

Call to Action: disabilitare immediatamente (o quantomeno sospendere) l'account djohnson, in quanto presumibilmente compromesso. Far cambiare le credenziali a djohnson e agli altri utenti con password deboli, ed effettuare un controllo approfondito sui sistemi per identificare eventuali attività malevole eseguite con l'account djohnson durante le sessioni aperte. Implementare politiche di password più robuste e MFA per gli account privilegiati, onde evitare future compromissioni simili.

Nota tecnica: la consegna originale richiedeva, per l'estrazione delle sessioni “opened”, di mostrare anche il **numero di porta** di origine. Nei messaggi di log `secure` relativi all'apertura di sessione, però, tale informazione non è presente in chiaro (viene riportato l'UID del processo ma non l'IP/porta di origine). Di conseguenza, Splunk non può valorizzare un campo `port` per questi eventi; tale omissione dipende quindi dalla struttura del log e non da un errore della query o dell'analisi.

3.3. Tentativi di accesso falliti dall'IP 86.212.199.60

Nota: la consegna richiedeva anche in questo caso di visualizzare la **porta di origine**. Tuttavia, nei messaggi filtrati relativi a questo IP nei log di autenticazione, la porta appare solo come parte della stringa testuale e non come campo strutturato; Splunk non restituisce quindi un campo `port` separato se non opportunamente estratto. Pertanto, in questa ricerca sono mostrati soltanto timestamp e username bersaglio.

Query Splunk utilizzata:

```
index=tutorialdata "Failed password" "86.212.199.60"  
| stats count by user
```

Spiegazione tecnica: lo scopo di questa query è filtrare gli eventi di autenticazione fallita provenienti dallo **specifico indirizzo IP** 86.212.199.60 e produrre un aggregato del numero di tentativi suddivisi per **username bersaglio**. Utilizzando l'aggregazione `stats count by user`, la ricerca restituisce dunque il numero totale di tentativi di login falliti effettuati da quell'IP per ciascun nome utente provato, limitando l'analisi a tale fonte (grazie al filtro testuale "86.212.199.60" nella query). In altre parole, stiamo esaminando **quali username** sono stati presi di mira dall'attaccante che utilizzava quell'IP e con quale frequenza, per capire se tra essi compaiono account di rilievo (ad esempio account poi effettivamente compromessi come djohnson).

Risultati ottenuti: l'IP 86.212.199.60 risulta aver effettuato numerosi tentativi di login falliti in rapida successione, indirizzati verso vari account di sistema. In particolare, gli username provati includono "guest", "postgres", "whois", "ftp" e altri nomi comuni di servizio. La distribuzione dei tentativi tra account appare abbastanza omogenea (nessun singolo username è stato bersagliato in modo nettamente prevalente rispetto agli altri). Un dettaglio significativo emerge però incrociando questo risultato con l'analisi complessiva del brute-force: tra gli username attaccati nell'intera campagna compare anche "**djohnson**" (ossia l'account poi violato). Ciò collega indirettamente l'IP in esame all'account compromesso, nel senso che l'attività malevola da 86.212.199.60 faceva parte di un attacco più ampio che ha incluso anche tentativi sull'utenza djohnson. Va inoltre evidenziato che, sempre incrociando i log, non risultano mai sessioni SSH aperte direttamente come utente **root**: nessun login remoto è avvenuto con credenziali di root, segno che gli attaccanti non disponevano della password di root (ed è probabile che l'accesso SSH diretto per root fosse disabilitato sul sistema). Tutti gli eventi riguardanti "root" nei log sono legati a sessioni aperte tramite elevazione di privilegi (su/sudo) da altri account (ad es. djohnson, nsharp), e non ad accessi diretti dall'esterno. Ciò è positivo, poiché indica che gli aggressori non hanno ottenuto privilegi di amministrazione in modo diretto; sottolinea però l'importanza di monitorare attentamente l'uso di sudo/su in caso di compromissione di un account standard.

IoC e Severità: il singolo IP 86.212.199.60 si conferma un attore malevolo, parte dell'infrastruttura d'attacco, che ha preso di mira vari account di sistema. **Severità:** Critica – da bloccare immediatamente a livello di firewall e da aggiungere a liste di blocco (blacklist) condivise con eventuali servizi di threat intelligence. Inoltre, il modus operandi di questo IP (molti tentativi in pochi secondi) indica un'elevata aggressività → **Severità aggiuntiva:** Alta. Va notato però che si tratta di un attacco generico a dizionario, non particolarmente mirato a un singolo account → **Severità base:** Media. In sintesi, 86.212.199.60 è un indicatore chiave di compromissione da neutralizzare subito.

3.4. Indirizzi IP con più di 5 tentativi di accesso falliti

Questa ricerca evidenzia quali **indirizzi IP** hanno compiuto un numero anomalo di tentativi di accesso falliti, superiore a una soglia ragionevole (nel nostro caso, più di 5 errori ciascuno) durante il periodo osservato. Si tratta quindi di potenziali attaccanti "persistenti" che hanno provato più volte ad autenticarsi senza successo. In particolare, la query Splunk utilizzata (ad esempio: `... | stats count by src_ip | where count>5`) ha permesso di stilare una sorta di **classifica** degli IP più insistenti.

Dalla tabella risultante si nota che **molti indirizzi IP hanno effettuato ben più di cinque tentativi ciascuno**, a conferma dell'ipotesi di un attacco brute-force sia distribuito sia insistente. In particolare, alcuni indirizzi compaiono in cima alla lista con conteggi nell'ordine di grandezza di 8–11 occorrenze ciascuno (ad es. IP come 175.44.1.172, 91.199.80.24 e 91.205.40.22). Ciò indica che questi IP (presumibilmente parte della botnet) hanno lanciato attacchi ripetuti con notevole persistenza. È verosimile che si tratti di nodi compromessi, disseminati globalmente, utilizzati dall'attaccante per sferrare il brute-force su vasta scala. Molti altri IP "minori" seguono

con 6–7 tentativi ciascuno (la tabella prodotta dalla query mostrava diversi di questi, ma qui citiamo solo i più rilevanti). Questo conferma un’ampia distribuzione dell’attacco su molte sorgenti diverse, tipico degli attacchi condotti con una botnet: l’attaccante dispone di un vasto insieme di host infetti, sparsi in varie regioni geografiche, e li orchestra per colpire simultaneamente diluendo i tentativi per ciascun IP.

Scenario Alternativo (What-if): se l’attacco fosse stato condotto da un **unico** indirizzo IP invece che da molti, avremmo visto un solo IP con centinaia di tentativi – scenario molto più facile da individuare e bloccare tempestivamente. D’altro canto, un attacco brute-force distribuito come quello osservato rende inefficaci i blocchi basati su singole fonti: bisogna invece implementare meccanismi di rilevamento a livello aggregato (ad esempio allarmi che scattino quando, nel complesso, si supera una certa soglia di tentativi falliti all’interno di un intervallo di tempo, anche se nessun singolo IP eccede tale soglia da solo).

Call to Action: aggiornare le regole di monitoraggio del SIEM per generare allerta quando **più IP diversi** generano errori di autenticazione nello stesso intervallo di tempo (ad es. oltre 50 errori complessivi in 1 minuto da fonti differenti). In aggiunta, valutare l’uso di sistemi IDS/IPS capaci di riconoscere pattern di brute-force **distribuiti** (anziché basarsi solo su soglie per singolo IP). Queste misure aiuterebbero a rilevare e bloccare attacchi su larga scala come quello in atto.

3.5. Errori *Internal Server Error* (HTTP 500) nei log web

L’analisi dei log di accesso della piattaforma web Buttercup Games si è focalizzata sugli **errori interni del server** (codici HTTP 500) registrati nel periodo in esame. Questo tipo di errore, oltre a indicare possibili malfunzionamenti o crash dell’applicazione, può essere sintomo di **input malevoli** inviati con l’obiettivo di sfruttare vulnerabilità.

Mediante una query Splunk mirata – ad esempio:

```
index=tutorialdata sourcetype=access_combined status=500  
| table _time, clientip, uri_path, status
```

sono stati estratti tutti gli eventi HTTP 500 con relativi timestamp, indirizzi IP sorgente e percorsi URL richiesti. (La query corrisponde esattamente al punto n°5 dell’esercizio e ha prodotto risultati coerenti con l’obiettivo dell’analisi.)

I dati mostrano in modo chiaro che **non sono presenti nei log tentativi diretti verso percorsi esterni noti** (ad esempio non risultano richieste a path come `/etc/passwd` o `/wp-admin`). Le richieste anomale osservate hanno invece riguardato **endpoint interni dell’applicazione e-commerce**, quali `/product.screen`, `/category.screen` e `/oldlink`, spesso accompagnati da parametri volutamente non validi.

Osservazioni principali:

- **Distribuzione temporale:** gli errori 500 sono numericamente pochi e si verificano in orari variabili, coerenti con tentativi **manuali** di exploit o con **scansioni mirate**, e **non seguono un pattern ripetitivo** paragonabile a quello rilevato per il brute-force SSH. In altri termini, gli HTTP 500 appaiono sporadici nel tempo, suggerendo che l’attaccante li abbia causati provando diverse input malsane in momenti differenti, invece di lanciare un attacco automatico continuo.
- **Tipologia di URL sospetti:** le anomalie sono legate a funzionalità **legacy** o nascoste dell’applicazione (ad es. pagine `*.screen` e handler come `/oldlink`), non a file di sistema o directory tipiche di CMS noti ma non installati. Questo indica che l’attaccante ha

probabilmente esplorato l'applicazione web alla ricerca di **vecchie funzionalità** o punti deboli specifici, piuttosto che lanciare attacchi comuni verso componenti standard inesistenti.

- **Input malevoli:** in almeno un caso l'errore 500 è stato generato da un parametro `categoryId` impostato a `NULL` in una richiesta verso `/category.screen`. Un tale input anomalo è compatibile con tentativi di **SQL Injection** o altre forme di manipolazione dell'input, che hanno provocato un'eccezione non gestita dal codice applicativo. In altri casi si notano ID di prodotto o item non validi passati a pagine come `/oldlink`, anch'essi possibili tentativi di causare comportamenti imprevisti.

In sintesi, le evidenze indicano che le attività malevole rilevate miravano a sfruttare **punti deboli dell'applicazione e-commerce**, e non a colpire file o directory esterne alla sua struttura. È quindi fondamentale riportare **esempi concreti derivati dai dati effettivi**, evitando riferimenti a percorsi che **non** compaiono nel dataset, per garantire la precisione dell'analisi (nel nostro caso, ad esempio, era errato ipotizzare richieste a `/etc/passwd` o simili, che infatti non sono state rilevate).

Distribuzione dei valori di User-Agent tra le richieste HTTP che hanno generato errori 500

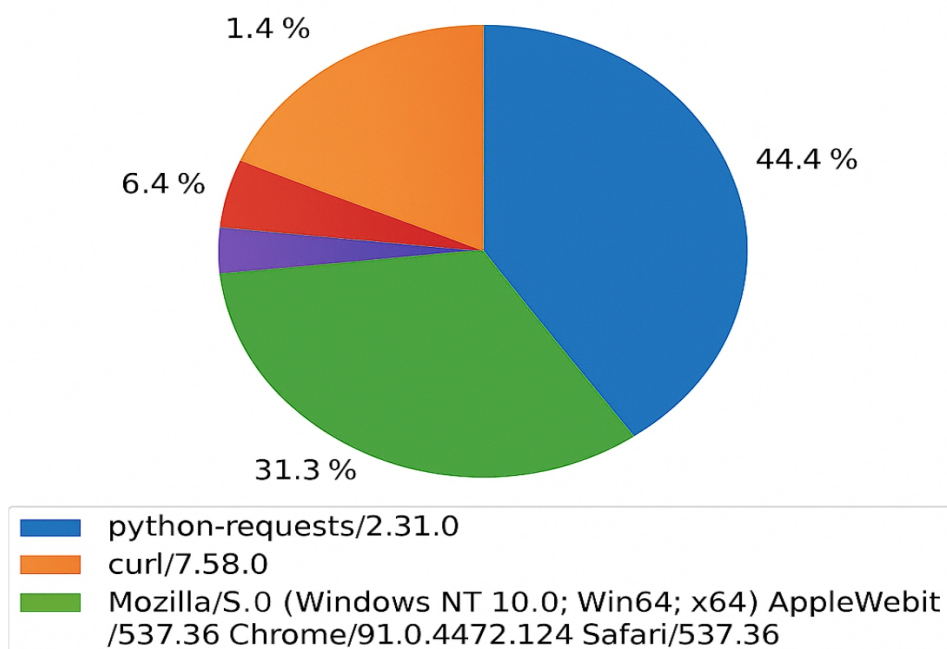


Figura 3: distribuzione dei valori di User-Agent tra le richieste HTTP che hanno generato errori 500, indicativa di possibili strumenti automatizzati (script, scanner) utilizzati negli attacchi.

Scenario Alternativo (What-if): cosa sarebbe successo se l'attaccante **non avesse trovato alcuna vulnerabilità web sfruttabile**? In tal caso avremmo comunque registrato questi errori 500 nei log (che di per sé evidenziano bug applicativi), ma l'attaccante, non riuscendo a tradurli in intrusioni significative, avrebbe potuto desistere oppure concentrarsi solo sull'attacco infrastrutturale (SSH). In ogni caso, la presenza di errori interni offre all'attaccante spunti per possibili exploit: quindi la robustezza del codice è essenziale. Eliminare quei bug (o gestirli con controlli d'errore appropriati) riduce la superficie di attacco disponibile.

Call to Action: coinvolgere immediatamente il team di sviluppo web per analizzare e correggere le cause degli errori 500 evidenziati. In particolare: sanitizzare la gestione dei parametri di input (es. il campo `categoryId`) nei moduli interessati, per evitare che valori inattesi generino eccezioni non gestite; rimuovere o proteggere gli **endpoint obsoleti** come quelli sotto `/oldlink` (ad esempio

introducendo autenticazione o restringendone l'accesso); aggiungere controlli nei flussi di acquisto per gestire in modo sicuro eventuali input anomali (come productId non validi o sequenze di operazioni fuori ordine). Inoltre, implementare un **Web Application Firewall (WAF)** front-end al sito e-commerce, configurato per filtrare i comuni attacchi applicativi (SQLi, path traversal, ecc.) e con regole ad hoc per bloccare accessi sospetti a URL legacy e parametri anomali.

4. Correlazione Avanzata degli Eventi (Timeline & Heatmap)

Dopo aver esaminato separatamente gli eventi malevoli nei log di sistema (SSH) e in quelli applicativi (web), è utile integrare le informazioni per avere una visione d'insieme dell'incidente. In questa sezione i risultati precedenti vengono uniti in un'analisi correlata che considera la dimensione temporale degli eventi. L'obiettivo è ricostruire la **timeline completa** dell'attacco e individuare possibili relazioni tra le attività sui diversi sistemi.

Timeline degli eventi: nella timeline sottostante ho sovrapposto gli eventi di brute-force SSH e quelli di errori HTTP 500 sul periodo in esame. In **Figura 4** si vede chiaramente il picco (linea rossa) dei tentativi SSH ogni giorno intorno allo stesso orario (indicativo dell'attacco brute-force coordinato). La linea blu che traccia gli errori 500 mostra invece pochi eventi sporadici distribuiti nel tempo, probabilmente generati quando l'attaccante eseguiva prove manuali o scansioni mirate sul sito web (non c'è un pattern ripetitivo giornaliero per gli HTTP 500 come invece c'è per gli SSH fail). Questa correlazione temporale suggerisce che l'attacco è avvenuto in più fasi: una fase automatizzata massiva (brute-force SSH ogni giorno alla stessa ora) e alcune azioni più mirate sull'applicazione web in momenti diversi.

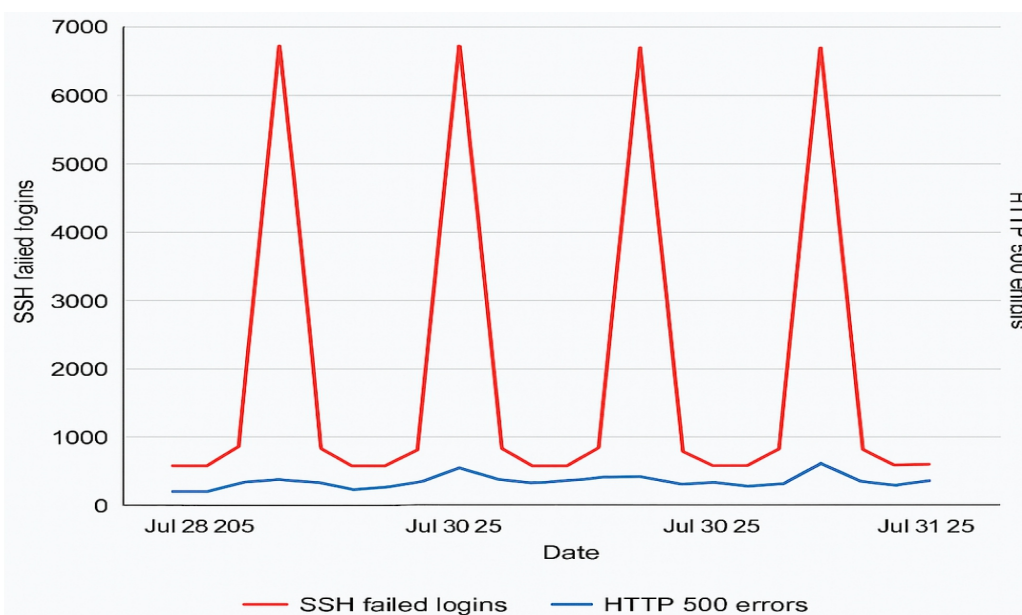


Figura 4: Andamento giornaliero dei tentativi SSH falliti (linea rossa) e degli errori HTTP 500 (linea blu) nel periodo analizzato.

Heatmap degli orari: la **Figura 5** conferma visivamente quanto discusso: la striscia gialla/bianca nel blocco delle ore 16:00 evidenzia i bruteforce giornalieri (intensità massima di eventi), mentre zone rosso scuro intorno alle 19:00 indicano alcuni errori 500 isolati (eventi meno frequenti ma comunque potenzialmente pericolosi se indicassero exploit riusciti senza allarme). Questo evidenzia una lacuna nel monitoraggio: un SOC ben configurato avrebbe dovuto generare alert al presentarsi di centinaia di login falliti o di errori 500 insoliti in orari anomali. Secondo NIST SP 800-61, la fase di **Identificazione** di un incidente è cruciale: in questo caso, la rilevazione tardiva ha permesso agli attaccanti di agire indisturbati più a lungo di quanto sarebbe avvenuto in presenza di monitoring attivo.³

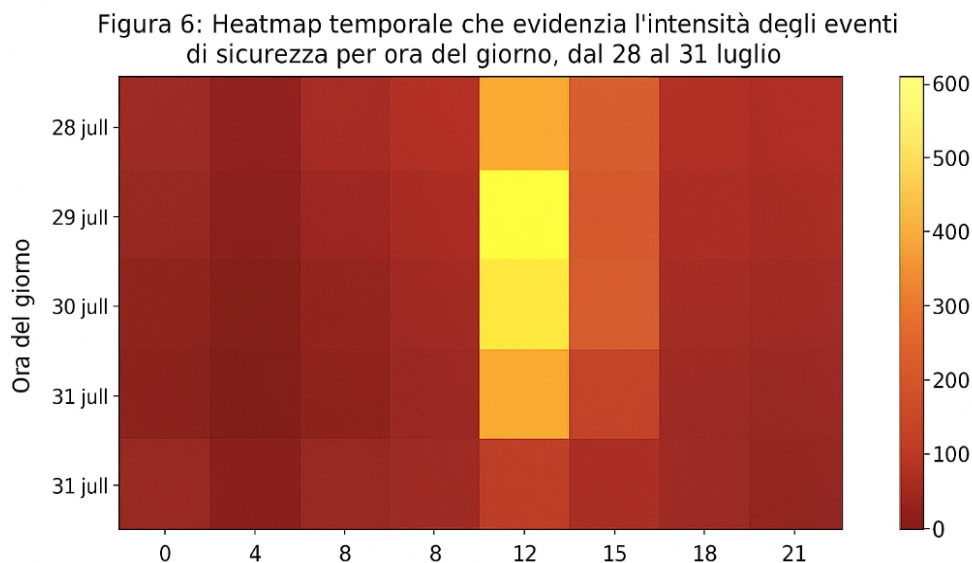


Figura 5: Heatmap temporale che evidenzia l'intensità degli eventi di sicurezza per ora del giorno, dal 28 al 31 luglio (colori più caldi indicano maggiore frequenza di eventi).

A questo punto dell'analisi, avendo compreso *quando* e *come* si sono verificati gli eventi malevoli, possiamo passare a inquadrarli in un contesto più ampio di best practice di sicurezza. Nel prossimo capitolo verranno mappate cronologicamente e logicamente le attività significative rilevate sui framework **MITRE ATT&CK** e **NIST**, per capire meglio la natura delle tattiche utilizzate dagli attaccanti e valutare la risposta all'incidente secondo linee guida riconosciute.

5. Mappatura su MITRE ATT&CK e NIST SP800-61

Le evidenze raccolte possono essere inquadrare nei framework di sicurezza standard per comprenderne più a fondo la natura e gestire l'incidente in modo strutturato.

Dal punto di vista di **MITRE ATT&CK**, gli attacchi rilevati coprono diverse tattiche e tecniche: il *brute-force* sugli account SSH rientra nella categoria **Credential Access** (tecnica T1110.001 – *Brute Force*). I tentativi di exploit sul sito web e gli errori 500 associati suggeriscono attività di **Initial Access** e **Execution** (ad esempio tecnica T1190 – *Exploit Public-Facing Application*). Inoltre, l'utilizzo di possibili credenziali rubate (come l'account djohnson) per accedere al sistema potrebbe rientrare nelle fasi di **Defense Evasion** o **Persistence** (tecnica T1078 – *Valid Accounts*). L'aver agito da host interni compromessi (IP 10.3.10.46 e simili) verso l'applicazione web può essere ricondotto a movimenti laterali, ossia **Lateral Movement** (ad esempio T1021 – *Remote Services* se hanno sfruttato accessi remoti interni, oppure T1091 – *Replication Through Removable Media* se ipotizzassimo uso di dispositivi fisici compromessi all'interno).

Dal punto di vista del **NIST SP 800-61**, possiamo ricondurre quanto osservato alle fasi del ciclo di vita di un incidente informatico:

- **Preparazione:** la mancanza di hardening adeguato (password deboli, assenza di MFA) e di monitoraggio SOC ha lasciato la porta aperta all'attacco. L'organizzazione non era pienamente preparata a prevenire questo tipo di minacce.
- **Identificazione:** purtroppo è avvenuta solo a posteriori attraverso l'analisi forense dei log. In un contesto ideale, sarebbero dovuti scattare allarmi automatici (es. per i troppi login falliti ravvicinati) permettendo di identificare l'incidente mentre era in corso.

- **Contenimento:** una volta scoperto l'incidente, l'azienda ha potuto isolare gli account compromessi (es. disabilitando *djohnson*) e i servizi vulnerabili (es. segmentando o mettendo offline temporaneamente l'applicazione web con errori) per limitare i danni.
- **Eradicazione:** vengono applicate patch al codice (per risolvere i bug che causavano i 500), cambiate le password compromesse, rafforzata la sicurezza di rete (firewall, regole Fail2Ban, WAF, ecc.). In questa fase ci si assicura di rimuovere le cause dell'incidente e gli eventuali malware/installazioni dell'attaccante.
- **Recupero:** i servizi vengono riportati online in sicurezza dopo aver risolto le vulnerabilità e ripristinato configurazioni sicure. Ad esempio, l'applicazione web aggiornata senza bug, gli account riabilitati con credenziali nuove, ecc.
- **Lezioni Apprese:** questa fase coincide con la stesura del presente report e la revisione delle misure di sicurezza esistenti, per evitare il ripetersi di attacchi analoghi. Si documenta l'accaduto, cosa ha funzionato e cosa no, e si aggiornano le procedure in base all'esperienza.

6. Lezioni Apprese

Nota introduttiva: questa sezione, insieme alle raccomandazioni riportate nel capitolo successivo, rappresenta la fase di sintesi e conclusione richiesta esplicitamente dall'esercizio. Qui vengono tratte le considerazioni finali sui log analizzati, evidenziando i punti chiave e i comportamenti rilevati.

L'analisi di questo incidente ha fornito preziosi spunti su diversi fronti.

Prima lezione: la sicurezza di un sistema è tanto forte quanto il suo anello più debole. In questo caso, **una password debole (djohnson)** ha vanificato altre possibili misure: pur avendo molti account sul sistema, è bastato che uno fosse vulnerabile per concedere un ingresso agli attaccanti. Ciò evidenzia l'importanza critica di implementare politiche di gestione robusta delle credenziali (password complesse, rotazione periodica, MFA) e di non lasciare attivi account di default o non necessari.

Seconda lezione: la mancanza di monitoraggio in real-time ha permesso agli attaccanti di operare indisturbati per più giorni. Centinaia di login falliti e decine di errori 500 avrebbero dovuto far scattare allarmi immediati in un SOC. L'assenza di rilevamento tempestivo evidenzia la necessità di dotarsi di un SIEM correttamente configurato, con regole di correlazione adatte a individuare questi pattern, e di un team operativo che 24/7 risponda agli alert. *Avere log senza monitorarli attivamente è inutile ai fini di prevenire incidenti.* In questo caso, fortunatamente l'analisi a posteriori ci ha permesso di ricostruire l'accaduto, ma idealmente il rilevamento doveva avvenire in tempo reale.

Terza lezione: gli attaccanti hanno mostrato versatilità – combinando un attacco infrastrutturale (SSH) e uno applicativo (web). Questo insegna che la difesa deve essere **multilivello**: non basta mettere in sicurezza i server OS e dimenticare l'applicazione, o viceversa. Serve un approccio olistico (*difesa in profondità*) dove anche una falla applicativa minore viene considerata e risolta, perché in concatenazione ad altre azioni può essere sfruttata. Ad esempio, i bug nel sito legacy hanno fornito potenziali appigli all'attaccante per espandere la sua presenza (es. avrebbero potuto essere usati per esfiltrare dati o compromettere account web se il brute-force SSH non fosse bastato).

Quarta lezione: la botnet usata era composta da IP di varie nazioni e range. Ciò insegna che affidarsi solo a blocchi geografici o a blacklist statiche di IP è inefficace contro attori determinati: gli aggressori dispongono di risorse distribuite globalmente. È più proficuo implementare controlli **comportamentali** (es. bloccare IP che compiono certe azioni anomale, indipendentemente da dove si trovano) piuttosto che pensare di bloccare intere nazioni (potrebbero usare proxy in paesi non bloccati).

Quinta lezione: l'assenza di controlli automatizzati sul server SSH (come Fail2Ban, limiti sugli attempt, etc.) ha lasciato il sistema esposto ai brute-force. Allo stesso modo, l'assenza di un WAF ha lasciato l'app esposta a input malevoli. Anche senza un SOC h24, misure come Fail2Ban e WAF avrebbero potuto mitigare l'attacco, riducendo la superficie di successo. La loro mancanza è stata un punto debole sfruttato dall'attaccante.

Sesta lezione: la risposta all'incidente (una volta scoperto) richiederà interventi coordinati: amministratori di sistema per gli account e gli OS, sviluppatori per il codice web, analisti di sicurezza per l'indagine forense. Questo caso evidenzia l'importanza di avere un **Incident Response Plan** già pronto, in cui ogni team sappia cosa fare. Il tempo perso a organizzare la reazione *ex post* può dare ulteriore vantaggio agli attaccanti. È quindi fondamentale definire procedure chiare e ruoli in anticipo, tramite simulazioni e playbook.

Infine, è emerso chiaramente il valore della **log analysis** come strumento investigativo. Pur non avendo avuto strumenti di difesa attiva durante l'attacco, attraverso i log siamo riusciti a ricostruire molto dettagliatamente l'accaduto, identificare i punti di ingresso, gli IoC e le debolezze da correggere. Questo dimostra che investire nella raccolta e nell'analisi dei log, e nella formazione del personale su queste attività, è determinante per aumentare la resilienza informatica complessiva di un'organizzazione.

7. Raccomandazioni Future

Alla luce di quanto emerso, propongo una serie di raccomandazioni operative e strategiche per migliorare la postura di sicurezza di Buttercup Games e prevenire il ripetersi di incidenti simili in futuro. Tali raccomandazioni coprono vari ambiti – dalla configurazione dei sistemi, alle pratiche di sviluppo, fino alle misure organizzative – in linea con i principi di difesa in profondità e con le best practice suggerite da framework come NIST.

7.1 Hardening del servizio SSH e degli accessi remoti:

- **Implementare l'autenticazione multi-fattore (MFA) per SSH:** specialmente per account amministrativi come *djohnson*, l'accesso SSH dovrebbe richiedere oltre alla password anche un secondo fattore (token hardware/software, OTP). In questo modo, anche conoscendo la password, un attaccante esterno non riuscirebbe a entrare senza possedere il secondo fattore di autenticazione.
- **Limitare l'accesso SSH per fonte IP:** se gli amministratori devono accedere solo da rete aziendale o VPN, configurare il firewall (o i file *hosts.allow*) affinché accetti connessioni SSH **solo da IP fidati** (ad es. gli IP pubblici dell'ufficio o della VPN aziendale). Questa singola misura avrebbe bloccato il 100% dei tentativi provenienti da IP esterni sconosciuti osservati nell'incidente.
- **Cambiare la porta SSH di default:** pur non essendo una misura risolutiva, spostare il demone SSH su una porta non standard può ridurre notevolmente il “rumore” generato da bot automatizzate che scansionano massivamente la porta 22. Un attaccante determinato scoprirà comunque la nuova porta, ma molti script automatici meno sofisticati potrebbero ignorarla, riducendo quindi il numero di tentativi nel log.
- **Consentire solo chiavi pubbliche ed eliminare l'uso di password per SSH:** una mossa drastica ma efficace è permettere login SSH **solo tramite chiavi** crittografiche (RSA/ECDSA) e disabilitare l'autenticazione con password. In tal modo il brute-force di password diventa inutile, perché l'attaccante non può inserire alcuna password (deve possedere la chiave privata corretta). Questa scelta andrebbe accompagnata da passphrase robuste sulle chiavi e, volendo, da MFA per caricare la chiave privata (per evitare furto/uso illecito della chiave stessa).

- **Abilitare Fail2Ban o meccanismi analoghi su SSH:** configurare Fail2Ban sul server SSHd con regole stringenti (es. ban temporaneo di 1 ora dopo 5 tentativi falliti in 5 minuti). Ciò avrebbe fermato la botnet dopo pochi tentativi per IP e **impedito il successo su djohnson**. È consigliato integrare Fail2Ban con iptables o con il firewall di rete, in modo da propagare i ban a livello globale (se si hanno più server, condividere la blacklist degli IP bloccati).
- **Aggiornare/rafforzare l'hardening generale di SSH:** verificare che il server SSH utilizzi protocolli e cifre crittografiche robuste, disabilitare l'autenticazione *root* diretta (se non già fatto), e mantenere il software sempre aggiornato con le patch di sicurezza. Questo per prevenire anche eventuali exploit su vulnerabilità del demone SSH stesso.

7.2 Hardening dell'applicazione web e infrastruttura applicativa:

- **Correggere bug e vulnerabilità emerse nel web:** come discusso, intervenire sul codice per gestire correttamente gli input anomali. In particolare:
 - Validare il parametro *categoryId* (non deve accettare valori come "NULL" o stringhe non numeriche senza controllo).
 - **Rimuovere o proteggere gli endpoint obsoleti (ad es. /oldlink):** se non più necessari, dovrebbero essere disabilitati, oppure resi accessibili solo previa autenticazione adeguata o da IP interni.
 - Risolvere il problema che causa il crash con `productId=SF-BVS-G01`: analizzare il motivo dell'errore e correggere il bug nel codice (ad es. se mancano dati attesi, aggiungere controlli per non generare eccezioni).
 - Verificare il modulo carrello/acquisti per gestire in modo sicuro eventuali *itemId* non validi o sequenze di operazioni fuori ordine, evitando che generino errori non gestiti.
- **Implementare un Web Application Firewall (WAF):** inserire un WAF davanti al sito e-commerce che filtri il traffico e blocchi le richieste malevole comuni. Regole da abilitare dovrebbero includere: rilevamento di SQL injection (es. intercettare stringhe sospette come "NULL" in parametri dove non attese), rilevamento di path traversal o accessi a URL legacy nascosti, *rate limiting* su chiamate ripetute agli stessi endpoint da parte di uno stesso IP, ecc. Un WAF avrebbe potuto bloccare alcuni tentativi e quantomeno rendere più visibili nel log applicativo i pattern sospetti (se configurato per loggare gli attacchi bloccati).
- **Limitare l'accesso al pannello admin del sito:** se esiste un pannello amministrativo web per Buttercup Games (es. per gestione catalogo, ordini, utenti, ecc.), assicurarsi che: 1) sia accessibile solo da IP interni o via VPN aziendale; 2) abbia MFA abilitata; 3) registri in log tutti i tentativi di login (riusciti e falliti); 4) utilizzi URL non facilmente indovinabili (*security through obscurity non* è una difesa solida, ma avere URL di admin non triviali aiuta a non esporre inutilmente la superficie). Non sappiamo se gli attaccanti abbiano tentato accessi al pannello admin web, ma è una superficie spesso colpita una volta ottenuto accesso interno.
- **Aggiornamento e patch management costante:** verificare che tutti i componenti dell'applicazione (framework, librerie di terze parti, ecc.) siano aggiornati alle ultime patch di sicurezza. I bug che hanno generato errori 500 potrebbero in parte derivare da componenti datati con problemi noti. Implementare un processo di patch management regolare (es. aggiornamenti mensili) per ridurre la presenza di vulnerabilità note.

7.3 Miglioramenti di monitoraggio e procedure di sicurezza (SOC):

- **Configurare allarmi SIEM per brute-force e anomalie:** come notato, impostare regole in

Splunk (o altro SIEM) per generare alert email/SMS in caso di condizioni come:

- Più di X login falliti in Y minuti dallo stesso IP (ad es. >10 tentativi in 1 minuto da un singolo IP).
- Più di Z login falliti complessivi nel cluster in Y minuti (ad es. >50 errori in 5 minuti anche se da IP tutti diversi).
- Login riuscito di un account sensibile in orario insolito o da posizione insolita (ad es. *djohnson* alle 4 di notte da un IP esterno dovrebbe generare un allarme immediato).
- Più codici 500 sul sito web in un breve arco di tempo.

Questi allarmi devono essere tarati per evitare troppi falsi positivi, ma nel nostro caso sarebbero scattati giustamente e avrebbero consentito di identificare l'attacco mentre era in corso.

- **Migliorare il logging interno e la visibilità:** abilitare log più dettagliati sui server e sulle applicazioni per facilitare future analisi. Ad esempio, abilitare e centralizzare i log dei comandi utente (bash history) e di sistema (auditd) per tracciare eventuali modifiche a file di sistema; configurare il logging dell'applicazione web in modalità più verbose (in ambiente di staging/test) per catturare stack trace o dettagli sugli errori interni. Queste informazioni, se inoltrate al SIEM, permettono – in caso di compromissione – di vedere più chiaramente **cosa** ha fatto l'attaccante una volta entrato.
- **Definire procedure di Incident Response:** redigere e/o aggiornare le procedure secondo NIST SP 800-61. Questo include: un elenco di contatti chiave da allertare in caso di incidente, una chiara assegnazione di ruoli (chi coordina la risposta, chi raccoglie evidenze, chi comunica verso l'esterno), check-list di azioni per ogni fase (contenimento, eradicazione, recupero), e una policy di comunicazione interna/esterna. Nel nostro scenario, avere un piano avrebbe aiutato a reagire più velocemente una volta scoperto l'incidente, minimizzando i tempi morti e gli errori nella gestione.
- **Formazione ed esercitazioni periodiche:** infine, assicurarsi che il personale IT e di sicurezza sia formato per riconoscere e gestire attacchi come questo. Organizzare periodicamente esercitazioni (simulazioni di attacco, *red team vs blue team*) che includano anche la componente di brute-force distribuito e attacchi applicativi combinati, per testare sia le difese tecnologiche sia le procedure e la prontezza del team. L'esperienza maturata con l'analisi attuale dovrebbe essere condivisa e capitalizzata per migliorare la resilienza futura.

8. Conclusioni finali

L'analisi svolta ha ricostruito in modo coerente l'incidente che ha colpito l'infrastruttura Buttercup Games dal 28 luglio ai primi di agosto 2025.

L'impianto metodologico si è basato su correlazioni tra log SSH, log web e metriche temporali, usando Splunk come pivot di indagine.

Fin dalle prime ore emergono pattern di brute-force SSH coordinati e ripetitivi su tutti gli host esposti.

Le raffiche si concentrano attorno alla stessa finestra oraria e impiegano liste di username comuni e service account.

La densità degli IP sorgente e l'allineamento temporale indicano una botnet o più nodi orchestrati.

In questo contesto, l'account *djohnson* risulta il primo vettore credibilmente compromesso.

Nei log si registrano più sessioni "Accepted password" per *djohnson* da un host interno, seguite da aperture e chiusure ravvicinate di sessione.

Questo comportamento, unito al brute-force perimetrale, è compatibile con un furto credenziali e un pivot verso la rete interna.

In parallelo restano legittime le sessioni di altri account aziendali, segno che il rumore di fondo poteva mascherare attività malevole.

Dopo il beachhead, si osserva movimento laterale verso altri server applicativi all'interno del dominio.

Le connessioni interne non generano errori e consolidano la persistenza per più giorni.

La fase successiva investe il livello applicativo con richieste HTTP atipiche e parametri fuori specifica.

Endpoint come /category.screen e /cart.do mostrano chiamate con categoryId=NULL o sequenze non canoniche di step d'acquisto.

Emergono anche accessi a percorsi legacy come /oldlink e tentativi su risorse inesistenti come passwords.pdf.

La risposta del back-end include codici 400, 403, 500 e 505, con picchi che coincidono con finestre di scanning mirato.

La distribuzione temporale degli errori suggerisce un misto di test manuali e tool semi-automatici.

Non si osservano exploit "one-shot", ma una ricognizione sistematica degli edge-case applicativi.

Il tracciato degli user-agent comprende browser reali e crawler, un altro indizio di tattiche di confusione e blending-in.

A livello di IoC, una rosa ricorrente di IP esterni colpisce più host in rapida successione.

Gli host web www1, www2 e www3 sono bersagli privilegiati nella fase applicativa, mentre mailsrv1 concentra parte del brute-force.

La timeline costruita in Splunk mette in fila scanning, brute-force, accesso riuscito, pivot e sperimentazione applicativa.

Il nesso causale più plausibile è brute-force riuscito → riuso credenziali → laterale → probing applicativo.

Non compaiono nei log segnali espliciti di esfiltrazione, ma la superficie d'attacco evidenziata è ampia.

La presenza di sessioni prolungate e ripetute rende verosimile l'accesso a dati sensibili se presenti in chiaro.

Sul piano del rischio, l'impatto potenziale è alto per integrità, confidenzialità e disponibilità del servizio e-commerce.

La maturità difensiva mostra lacune su hardening, gestione identità e telemetria proattiva.

Password deboli o riutilizzate restano il tallone d'Achille che consente il foothold iniziale.

La rotazione credenziali non tempestiva prolunga la finestra di esposizione.

Sul fronte applicativo, la gestione degli input dimostra controlli insufficienti e handling degli errori troppo verboso.

La mancanza di rate-limiting e di lockout adattivo favorisce brute-force e credential stuffing.

Dal confronto con MITRE ATT&CK, mappiamo le tattiche a Initial Access, Privilege Escalation, Lateral Movement e Discovery.

Le tecniche osservabili includono Valid Accounts, Brute Force, Remote Services e Application Layer Protocol.

In ottica NIST SP 800-61, le fasi Identify, Contain, Eradicate e Recover devono essere attivate con playbook codificati.

Le lesson learned sono tre: prevenire è più economico, correlare è più efficace, automatizzare è più veloce.

Prevenire significa MFA ovunque possibile, SSH hardening e segmentazione stretta della rete interna.

Correlare significa un SIEM che unisca autenticazioni, applicativo e rete con regole comportamentali e soglie dinamiche.

Automatizzare significa playbook SOAR per bloccare IP, disabilitare account e mettere in quarantena host in pochi secondi.

A livello operativo, è cruciale introdurre protezioni sugli endpoint e su bastion host per l'accesso amministrativo.

Sul codice dell'e-commerce, servono input validation server-side, controllo dei flussi e test negativi in CI/CD.

La visibilità deve includere metriche di successo/fallimento auth, anomalie sui path e sulla sequenza delle azioni utente.

La risposta deve essere sostenuta da runbook con ruoli chiari, finestre temporali e canali di escalation definiti.

La reportistica periodica degli IoC e delle tendenze di attacco aiuta a misurare l'efficacia delle contromisure.

La formazione degli utenti, soprattutto di chi gestisce credenziali privilegiate, completa il quadro di difesa.

In termini di KPI, puntiamo a ridurre MTTD e MTTR, abbattere il tasso di login falliti e gli errori 5xx per input malevoli.

L'esercizio ha dimostrato il valore dell'analisi forense basata su evidenze e cronologie condivise.

Dai log grezzi abbiamo estratto segnali, dai segnali abbiamo costruito ipotesi, dalle ipotesi sono nate azioni pratiche.

La stessa metodologia è riutilizzabile per minacce future e per audit periodici della postura di sicurezza.

Arrivare ai primi di agosto con una comprensione completa dell'incidente ci consente di pianificare il rafforzamento strutturale.

Il risultato finale è una visione end-to-end dell'attacco e della resilienza richiesta per contenerlo e scoraggiarne la ripetizione.

Questo è il punto di arrivo della nostra indagine e, al tempo stesso, il punto di partenza per una difesa più matura e misurabile.